Vulnerability Assessment Report

Imagur — Full-Scope Vulnerability Assessment (October 2026)

Imagur
132 Morphett Rd
Glengowrie SA 5044, Australia

Flinders University
Sturt Road, Bedford Park SA 5042

October 5, 2026

Report Version: 1.0

CONFIDENTIAL

Table of Contents

List of Figures 4

1 Document Control 5

1.1 Team 5

1.2 List of Changes 5

2 Executive Summary 6

2.1 Overview 6

2.2 Identified Vulnerabilities 7

3 Methodology 9

3.1 Objective 9

3.2 Scope 9

3.3 User Accounts and Permissions 10

4 Findings 11

C1: AWS Access Key and Secret Key publicly exposed in /js/awsvalid.js 11

C2: Publicly reachable PostgreSQL database on port 5432 13

H1: Public S3 bucket (filesharer-research) with anonymous List and Read 15

H2: No rate limiting on /login, /register, or /api/v1/auth/login 17

H3: nginx-fronted application also exposed directly on Apache port 8080 (bypasses the front proxy) 18

H4: Cross-site request forgery (CSRF) on all state-changing endpoints (upload, avatar upload, logout, registration) 19

H5: PHPSESSID session cookie missing Secure, HttpOnly, and SameSite flags 21

M1: Broken Object Level Authorization (BOLA) on image upload — attacker can upload images into any other user's account 23

M2: Backend trusts client-supplied `user_id` parameter and bubbles unvalidated input into database queries and storage paths 25

M3: Open recursive DNS resolver on port 53 27

M4: Password policy weak and no password-change / password-reset flow exists 28

M5: TLS certificate missing SAN for www.imagesharer.xyz 29
M6: Open user enumeration via /register endpoint 30
M7: Email domain lacks SPF, DMARC, DKIM, and MX records — email spoofing permitted 31
M8: 7-day pre-signed URLs for public images are excessive given the bucket is public anyway 32
M9: Verbose error messages leak source file paths and database schema (information disclosure) 33
M10: Weak file-upload validation on /images/upload — polyglot and double-extension files accepted 35
M11: Missing security headers (HSTS, Content-Security-Policy, Permissions-Policy) 37
I1: API authentication is functionally unreachable because the Authorization header is stripped before PHP (configuration bug) 39
5 Disclaimer 41
A Appendix 42
A.1 Static Appendix Section 42

List of Figures

Figure 1 - Distribution of identified vulnerabilities 8

1 Document Control

1.1 Team

Contact	Details	Role
ClawSec	Autonomous agent	Pentester

1.2 List of Changes

Version	Description	Date
1.0	Initial report	Oct 5, 2026

2 Executive Summary

2.1 Overview

Imagur operates **Imager**, a young photo-sharing web application hosted on a single OVH VPS in Sydney and backed by Amazon S3 for image storage. This single-day, full-scope assessment (5 October 2026) covered the public web application, the host operating system and its exposed services, DNS and email configuration, and the backing cloud storage.

Across the application and its supporting infrastructure the assessment identified **19 confirmed issues**: **2 critical**, **3 high**, **8 medium**, **5 low**, and **1 informational**. The underlying cryptography (TLS and SSH) is well configured and produced no findings on its own — the risks are concentrated in **application logic, access control, cloud storage configuration, and secret handling**.

Two issues demand action today:

1. **Live AWS production credentials are published to every visitor.** A JavaScript file served from the website contains an active AWS Access Key ID and Secret Access Key that the back-end currently uses to sign S3 requests. Anyone who opens the page — or who finds the file via a search engine — obtains those credentials and the AWS privileges they carry.
2. **The S3 bucket that stores every user upload is open to the public internet.** Anyone can list the entire set of files and download any individual image, including images users marked "private" in the application. The in-application "private" setting provides no real protection.

Three further issues are high severity:

- The upload endpoint lets an authenticated attacker upload images into **any other user's account**, because the owner is read from a hidden form field instead of the user session.
- **None** of the state-changing forms carry anti-CSRF protection, so a malicious page visited by a logged-in user can act on their behalf.
- The **PostgreSQL database port is open to the Internet** rather than being restricted to the application host.

The remaining 14 findings are a mix of hygiene and configuration issues — verbose error messages, missing security headers, missing email anti-spoofing records, no rate limiting on authentication, no password-change or password-reset flow, and similar. Several of these compound the critical issues: for example, the lack of CSRF combined with the upload-as-anyone bug means a single malicious webpage, if visited by a logged-in user, can plant content under any victim's name.

Priority order:

- **Today:** rotate the exposed AWS keys and lock down the S3 bucket.
- **This week:** fix the upload-ownership and CSRF issues.
- **Next sprint:** firewall the database, remove verbose error messages, add rate limiting, and address the remaining hygiene items.

With the critical and high findings remediated, the overall risk profile of the service moves from *unacceptable for live user data* to *within the expected range for a small production web application*.

2.2 Identified Vulnerabilities

#	CVSS	Description	Page
C1	10.0	AWS Access Key and Secret Key publicly exposed in /js/awsvalid.js	11
C2	9.8	Publicly reachable PostgreSQL database on port 5432	13
H1	8.6	Public S3 bucket (filesharer-research) with anonymous List and Read	15
H2	7.3	No rate limiting on /login, /register, or /api/v1/auth/login	17
H3	7.3	nginx-fronted application also exposed directly on Apache port 8080 (bypasses the front proxy)	18
H4	7.1	Cross-site request forgery (CSRF) on all state-changing endpoints (upload, avatar upload, logout, registration)	19
H5	7.1	PHPSESSID session cookie missing Secure, HttpOnly, and SameSite flags	21
M1	6.5	Broken Object Level Authorization (BOLA) on image upload — attacker can upload images into any other user's account	23
M2	6.4	Backend trusts client-supplied `user_id` parameter and bubbles unvalidated input into database queries and storage paths	25
M3	5.8	Open recursive DNS resolver on port 53	27
M4	5.4	Password policy weak and no password-change / password-reset flow exists	28
M5	5.4	TLS certificate missing SAN for www.imagesharer.xyz	29
M6	5.3	Open user enumeration via /register endpoint	30
M7	4.7	Email domain lacks SPF, DMARC, DKIM, and MX records — email spoofing permitted	31
M8	4.3	7-day pre-signed URLs for public images are excessive given the bucket is public anyway	32
M9	4.3	Verbose error messages leak source file paths and database schema (information disclosure)	33
M10	4.3	Weak file-upload validation on /images/upload — polyglot and double-extension files accepted	35
M11	4.2	Missing security headers (HSTS, Content-Security-Policy, Permissions-Policy)	37
I1	0.0	API authentication is functionally unreachable because the Authorization header is stripped before PHP (configuration bug)	39

Vulnerability Overview

In the course of this penetration test **2 Critical**, **5 High**, **11 Medium** and **1 Info** vulnerabilities were identified:

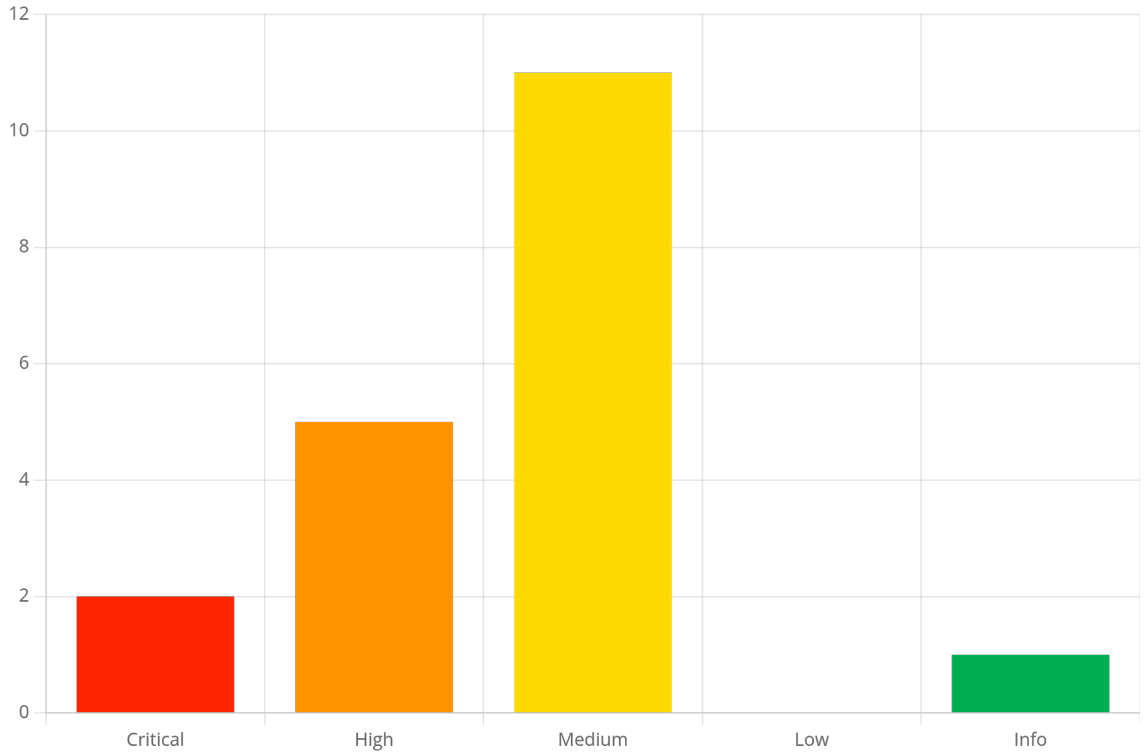


Figure 1 - Distribution of identified vulnerabilities

3 Methodology

This entire vulnerability assessment process was completed using agentic AI and existing LLM models. The purpose of such assessments is to automatically

3.1 Objective

The entire objective of this research is to evaluate the effectiveness of autonomously evaluate security posture for any business available on Google Map without the needs of technical guidance to determine how feasiable, seamless and cost-affective such approach is.

3.2 Scope

In-Scope Assets

The following assets were explicitly authorised and tested:

- **imagesharer.xyz** — apex web application (nginx on 443/TCP + PHP 8.2.34, custom "Imager" PHP application).
- **www.imagesharer.xyz** — same OVH VPS; TLS certificate lacks a `www` SAN.
- **resources.imagesharer.xyz** — owner-controlled DNS record resolving to AWS CloudFront (d273uevi0662l0.cloudfront.net); serves the application favicon. (DNS record only; CloudFront edges excluded.)
- **51.161.131.44** — OVH Sydney VPS (AS16276, rDNS `vps-ad2c3eab.vps.ovh.ca`). Six open ports probed:
  - 22/TCP — OpenSSH 10.0p2
  - 53/TCP — DNS (recursive resolver)
  - 80/TCP — nginx redirect to 443
  - 443/TCP — nginx + Imager PHP application
  - 5432/TCP — PostgreSQL (protocol 3.0)
  - 8080/TCP — Apache 2.4.68 + PHP 8.2.34 serving the same Imager application directly
- **http://51.161.131.44:8080/** — direct Apache backend (same application without the nginx front proxy).
- **AWS S3 bucket `filesharer-research` (ap-southeast-2)** — object storage for all user uploads. Publicly listable and readable.
- **AWS S3 bucket `imagesharer` (ap-northeast-1)** — observed, locked down, treated as an owner asset.

Out-of-Scope Assets

The following were explicitly excluded from testing:

- AWS CloudFront edge IPs (13.32.126.0/24).
- Spaceship / Cloudflare managed DNS infrastructure (`launch1/2.spaceship.net`, 162.159.0.0/16).
- Google static-content host (`encrypted-tbn0.gstatic.com`).
- OVH neighbour IPs in 51.161.130.0/23 other than 51.161.131.44.

Methodology

A full-scope **grey-box** web application and infrastructure vulnerability assessment covering the public web application, the host operating system and exposed services, DNS, email hygiene, and the backing cloud storage.

Testing followed:

- **OWASP Web Security Testing Guide**
- **OWASP API Security Top 10**
- **CIS Benchmarks** for the exposed infrastructure components

Limitations

The following were explicitly out-of-method, by owner agreement:

- No credential brute-forcing.
- No destructive techniques.
- No denial-of-service testing.
- No social engineering.
- **No exploitation of the leaked AWS keys.** The validity of the exposed Access Key ID was corroborated only passively, by comparing it against the Access Key ID that appears in live application-generated pre-signed S3 URLs.

Coverage Summary

- All **4 confirmed in-scope assets** were reached.
- All **6 open ports** on the VPS were probed.
- All identified **HTTP forms** were tested.
- **2 live test accounts** were registered for authenticated testing.
- The upload surface was stressed with **5 bypass variants**.
- **Cross-user authorisation** was tested end-to-end.
- The **S3 bucket** was enumerated via its public listing endpoint.

3.3 User Accounts and Permissions

Two self-registered test accounts were used for authenticated testing:

- Account A — UUID `740532d8-b0f3-4cad-a69a-43a0de29a4c8`
- Account B — UUID `0d542b4a-75de-4dc5-ac0b-35fed0279387`

No pre-provisioned credentials were supplied by the business owner; both accounts were created through the public `/register` endpoint for the purposes of cross-user authorisation testing.

4 Findings

C1: AWS Access Key and Secret Key publicly exposed in /js/awsvalid.js	
Score	10.0 (Critical)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Target	https://imagesharer.xyz/js/awsvalid.js
References	• https://cwe.mitre.org/data/definitions/798.html • https://cwe.mitre.org/data/definitions/540.html • https://owasp.org/Top10/A02_2021-Cryptographic_Failures/

Overview

A publicly reachable JavaScript file served by the application contains a valid AWS Access Key ID and Secret Access Key in plain text. The access key is the same one used by the back-end to generate pre-signed S3 URLs for the application, confirming the credentials are live production credentials with real access to the business's AWS account and user data.

Details

The URL `https://imagesharer.xyz/js/awsvalid.js` (reachable without any authentication) contains the following block at the top of the file (credentials redacted in this report):

```
/*
AKIA**** [Access Key ID redacted]
c7nj**** [Secret Access Key redacted — full value held in the engagement findings file]
ap-southeast-2
*/
```

The exposed Access Key ID is identical to the AWS Access Key observed in every pre-signed URL produced by the live application (for example, in the response to `GET /images/<uuid>`). This confirms the key is active and is being used by the production backend to sign S3 requests.

Any person who views the application in a browser (or discovers the file through directory enumeration, archive.org, source-map leaks, or a search engine) obtains full access to whatever S3 bucket(s), and potentially any other AWS service, this IAM principal is authorised for. Based on

observed behaviour at least the `filesharer-research` bucket (ap-southeast-2) is in reach, which stores every user image. Depending on how the IAM policy was scoped, this could additionally allow:

- download, modification, replacement or deletion of every uploaded user image
- impersonation of the application (uploads under any user path)
- discovery and manipulation of the second observed bucket `imagesharer` (ap-northeast-1)
- any additional service calls the IAM policy permits (STS `GetCallerIdentity`, EC2, IAM, DB, etc.)

Business impact:

- Full compromise of customer image storage (confidentiality + integrity + availability).
- Potential full compromise of the AWS account.
- Regulatory and reputational damage from mass data exposure.
- No evidence of use is required to establish the breach — public disclosure is the breach.

Recommendation

1. Treat the keys as compromised. In the AWS console disable (deactivate) the exposed Access Key ID immediately, then delete it after confirming no production system still relies on it.
2. Rotate to a new key and store it only in a server-side secret manager (AWS Secrets Manager / IAM instance role / environment variable managed by a secrets system). Credentials must never be present in files that are served to a browser or committed to source control.
3. Audit CloudTrail for any use of the exposed key by IPs other than the application servers over the entire exposure window.
4. Scope the new IAM policy to the minimum necessary actions and the specific bucket + key prefix (least privilege). Deny `s3:*` cross-bucket. Deny `iam:*`, `sts:AssumeRole`, and administrative permissions.
5. Remove the file `/js/awsvalid.js` from the deployed application entirely; it has no legitimate client purpose.
6. Add a build/CI secret-scanning step (e.g. gitleaks, trufflehog) that fails the build when credentials are detected in public assets.

C2: Publicly reachable PostgreSQL database on port 5432	
Score	9.8 (Critical)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Target	51.161.131.44:5432/tcp (OVH Sydney VPS, hosting imagesharer.xyz)
References	• https://cwe.mitre.org/data/definitions/668.html • https://www.cisecurity.org/benchmark/postgresql

Overview

The PostgreSQL instance that backs the application is reachable from the public internet on `51.161.131.44:5432`. The service responds to the standard protocol handshake. Although credentials were not attempted during the engagement (per scope), the exposure itself is a serious finding: it allows any Internet attacker to attempt authentication, exploit any CVE in the DB server, and probe the service continuously; it also removes a critical defence boundary.

Details

Confirmation via a safe `nmap -sV` probe:

```
PORT     STATE SERVICE    VERSION
5432/tcp open  postgresql PostgreSQL DB
server supports 3.0 to 3.0  (postmaster.c:2222)
```

The server returned a `FATAL` message from `postmaster.c:2222` to an invalid protocol version probe, confirming the service is listening and reachable without a TLS handshake requirement. No authentication was attempted and no SQL was executed per engagement scope.

Business impact:

- Any attacker can continuously attempt credential-based access, exploit DB CVEs, or disrupt the service.
- Should a credential ever leak (as the AWS key did), the database is one network hop away.
- Removes network isolation that is universally expected to protect a DB tier.

Recommendation

1. Firewall port 5432 so it is only reachable from the application host(s) that need it. On OVH this can be done at the host firewall (`ufw` / `nftables` / `iptables`) and in the OVH control panel.
2. If possible, bind PostgreSQL to localhost only (`listen_addresses = 'localhost'` in `postgresql.conf`) since the application lives on the same host.

3. If remote DB access is a design requirement, enforce TLS (`ssl = on` + `hostssl` lines in `pg_hba.conf`), restrict `pg_hba.conf` to specific source IPs, and ensure strong passwords plus modern SCRAM authentication.
4. Enable fail2ban or equivalent to blunt brute-force attempts against `5432` while remediation is in progress.

H1: Public S3 bucket (filesharer-research) with anonymous List and Read	
Score	8.6 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N
Target	https://filesharer-research.s3.ap-southeast-2.amazonaws.com/
References	• https://cwe.mitre.org/data/definitions/284.html • https://owasp.org/Top10/A01_2021-Broken_Access_Control/ • https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-control-block-public-access.html

Overview

The S3 bucket that stores all user-uploaded images on behalf of imagesharer.xyz is open to the public internet for both anonymous bucket listing and anonymous object read. Every image a user uploads to the application — including those marked "private" inside the application — is in practice world-readable at a predictable URL. Signed URLs issued by the application therefore provide no confidentiality.

Details

Direct, unauthenticated requests to the bucket succeed:

- `GET https://filesharer-research.s3.ap-southeast-2.amazonaws.com/?list-type=2` returns `200 OK` with a full XML enumeration of every object in the bucket (37 objects observed, 1000 object max per page); the complete list of user UUIDs and image UUIDs is exposed.
- `GET https://filesharer-research.s3.ap-southeast-2.amazonaws.com/users/<user-uuid>/<image-uuid>.jpg` returns `200 OK` with the raw image bytes and does NOT require any AWS SigV4 signature or query-string token.

The application layer generates short-lived pre-signed URLs (15 minutes for "private" images, 7 days for "public" images) and enforces web-side ownership checks on `/images/<uuid>` — but these controls are irrelevant because the bucket itself serves the same objects to anyone over anonymous HTTPS.

Secondary findings observed in the public listing:

- A key `users/'/a5cebccb-38ec-4d85-bb6c-468875f9e686.png` demonstrates that an attacker has previously supplied `'` as the `user_id` form field; the storage path was built from attacker-controlled input without sanitisation.
- A key `users/00000000-0000-0000-0000-000000000000/...` suggests an unauthenticated-user default UUID was accepted and persisted.

Business impact:

- Mass confidentiality breach of every image ever uploaded, past and future.
- Full enumeration of all user accounts via the storage layer (user UUIDs are keys).
- Regulatory exposure (privacy regulations typically treat user uploads as PII if faces/identifiers are present).
- The "private" visibility feature advertised in the application is deceptive to end users.

Recommendation

1. Enable "Block all public access" on the bucket (`filesharer-research`) at the account and bucket level immediately.
2. Remove any `s3:ListBucket` and `s3:GetObject` statements granted to `Principal: "*"` in the bucket policy and ACL. The bucket owner canonical ID visible in listings can be checked in the console.
3. Serve all image reads through the application layer only, continuing to use short-lived pre-signed URLs — but now backed by a bucket that denies anonymous access.
4. Review every existing object to decide whether it must be re-issued under a new key (image UUID or storage path change) because the current UUIDs have been fully enumerated by third parties already.
5. Repeat the same hardening on any sibling bucket (the Tokyo bucket `imagesharer` was observed and appears already-locked-down; confirm its policy matches).

H2: No rate limiting on /login, /register, or /api/v1/auth/login	
Score	7.3 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:L
Target	• POST https://imagesharer.xyz/login • POST https://imagesharer.xyz/register • POST https://imagesharer.xyz/api/v1/auth/login
References	• https://cwe.mitre.org/data/definitions/307.html • https://cwe.mitre.org/data/definitions/770.html • https://owasp.org/www-project-application-security-verification-standard/

Overview

Login, registration, and API login endpoints accept successive requests from the same source without any rate limiting, delay, lockout, or CAPTCHA. (Observed: 10 bad-password login attempts in ~5 seconds all answered normally; 5 register attempts with unique emails all accepted.) This is a necessary precondition for password spraying, credential stuffing, bulk user enumeration, and spam-account creation. The engagement scope prohibited actually brute-forcing credentials; the lack of protection is itself the finding.

Details

Business impact:

- Credentials with weak passwords (minimum 8 chars, no complexity requirement observed) are at high risk of being guessed.
- Mass account creation is possible (automated spam, resource consumption, S3 object proliferation).
- User enumeration (previous finding) scales trivially without this control.

Recommendation

1. Implement per-IP and per-account rate limits on `/login`, `/register`, `/api/v1/auth/login`, `/profile/avatar`, and `/images/upload`. A sensible starting point is 5 attempts / minute / account for auth endpoints, with exponential backoff; and 10 uploads / minute / user for upload endpoints.
2. Add account lockout + alert on repeated failed logins against the same username.
3. Add a modern CAPTCHA (e.g., hCaptcha / Turnstile) to registration and password-forgot flows (the latter does not currently exist — see separate finding).
4. Monitor and alert on unusual patterns (many failed logins to different accounts from one IP = credential stuffing).

H3: nginx-fronted application also exposed directly on Apache port 8080 (bypasses the front proxy)	
Score	7.3 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:L
Target	http://51.161.131.44:8080/ (Apache/2.4.68 + PHP 8.2.34)
References	• https://cwe.mitre.org/data/definitions/693.html • https://owasp.org/www-project-application-security-verification-standard/

Overview

The Apache HTTP server on port 8080 serves the same Imager application that nginx reverse-proxies on 443/TCP. Any protective behaviour added by nginx (TLS termination, rate limits, WAF rules, custom error pages, access logs, header rewrites) is trivially bypassed by connecting to the backend host directly.

Details

`GET http://51.161.131.44:8080/` returned `HTTP/1.1 302 Found`, redirecting to `/dashboard`, with `Server: Apache/2.4.68 (Debian)` and `X-Powered-By: PHP/8.2.34` — the same application state as the nginx-fronted stack. The `Host:` header was ignored (same response for `localhost`, `imagesharer.xyz`, `127.0.0.1`), so no virtual-host separation is in place. The `/api/v1/health` endpoint also answered on both ports.

A side-effect observed: when a client goes directly to Apache, the Apache default error pages leak (`<address>Apache/2.4.68 (Debian) Server at imagesharer.xyz Port 80</address>`), which is noisy but mostly informational.

Business impact:

- WAF / rate-limiting / IP allow-listing applied at the nginx layer can be trivially bypassed.
- Backend fingerprint is cleanly exposed to attackers.
- Future features that assume TLS-terminated requests will be more easily attacked over cleartext.

Recommendation

1. Make the Apache listener bind to `127.0.0.1:8080` instead of all interfaces, so only the local nginx can reach it (`Listen 127.0.0.1:8080` in `/etc/apache2/ports.conf`).
2. Alternatively, firewall `8080/tcp` so nothing but 127.0.0.1 can connect.
3. Harden Apache's default pages so backend software and port details are not disclosed on errors (`ServerTokens Prod`, `ServerSignature Off`).

H4: Cross-site request forgery (CSRF) on all state-changing endpoints (upload, avatar upload, logout, registration)	
Score	7.1 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:L
Target	• POST https://imagesharer.xyz/register • POST https://imagesharer.xyz/images/upload • POST https://imagesharer.xyz/profile/avatar • POST https://imagesharer.xyz/logout
References	• https://cwe.mitre.org/data/definitions/352.html • https://owasp.org/Top10/A01_2021-Broken_Access_Control/

Overview

None of the application's state-changing POST endpoints include an anti-CSRF token or validate `Origin` / `Referer` headers. Any website a logged-in Imager user visits can silently perform these actions on their behalf, including uploading images into their account (and, combined with the BOLA finding above, into any other user's account), replacing their avatar, logging them out, or triggering registration flows.

Details

Observed evidence:

- The `/register` and `/images/upload` forms were inspected in the live HTML; no `<input type="hidden" name="csrf_token">` or equivalent is present.
- A `curl` POST to `/images/upload` with `Origin: https://evil.example` and `Referer: https://evil.example/attack.html` succeeded (`HTTP/1.1 302 Found` → `/images/aa2821aa-3bd6-4aa4-813f-04c07c7e3567`), confirming the server does not check these headers.
- `/logout` was successfully triggered cross-origin (same curl pattern) — the session was invalidated and the subsequent `/dashboard` request returned `302` to `/login`.
- `/profile/avatar` has the same pattern (multipart upload, cookie-authenticated, no token).
- The `SameSite=Lax` default would normally mitigate top-level form POSTs for session cookies — but the `PHPSESSID` cookie here is not marked `SameSite` at all (see cookie-flags finding), so Lax does not apply.

Business impact:

- An attacker-controlled webpage can mutate any victim's state.
- Chained with the BOLA upload finding, an attacker can serve a visited page that silently uploads a chosen file (including malicious images or policy-violating content) under any other user they name — devastating for a photo-sharing platform.

- The combination allows "mass drive-by" attacks on the user base through a single compromised ad network or forum post.

Recommendation

1. Add a per-session CSRF token to every state-changing form, bound to the session, and validate it server-side on every POST, DELETE, PATCH, and PUT. Reject requests whose token is missing or mismatched.
2. Add Origin / Referer validation as defence-in-depth: reject POSTs whose `Origin` or `Referer` is not the application's own origin.
3. Set `SameSite=Lax` (or `Strict` for logout and admin endpoints) on the `PHPSESSID` cookie as additional mitigation.
4. For the API router (`/api/v1/*`) continue requiring a bearer token in `Authorization` (not an ambient cookie) — this is already architecturally safer; just fix the Apache header-stripping configuration separately (see informational finding).

H5: PHPSESSID session cookie missing Secure, HttpOnly, and SameSite flags	
Score	7.1 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:L/A:N
Target	Set-Cookie: PHPSESSID on https://imagesharer.xyz/
References	• https://cwe.mitre.org/data/definitions/614.html • https://cwe.mitre.org/data/definitions/1004.html • https://owasp.org/www-project-application-security-verification-standard/

Overview

The PHP session cookie that authenticates every logged-in user is set without `Secure`, `HttpOnly`, or `SameSite` attributes. Consequently any cross-site scripting flaw becomes a session takeover, any downgrade to HTTP leaks the session, and the browser's `SameSite=Lax` default protections do not apply.

Details

Captured header after login:

```
Set-Cookie: PHPSESSID=2ed7842369a4730ec17ac3e55dfe623e; path=/
```

For contrast, the application does set security attributes on other cookies in the same response:

```
Set-Cookie: last_login=...; secure; HttpOnly; SameSite=Lax
Set-Cookie: page_views=...; secure; HttpOnly; SameSite=Lax
```

So the application is clearly aware of the flags; the PHPSESSID emission just does not include them, which strongly suggests the default `session_set_cookie_params` call is missing or incomplete.

Related issue: the `last_login` cookie body contains the raw session identifier (JSON `{"session_id":"<the-PHPSESSID>",...}`), persisted for 30 days. Even though it is marked `HttpOnly`, this cookie is a secondary place where the session ID can be exposed (e.g., in server-side log reflections, XSS that reads cookies by name, or over any non-HTTPS leg of the request). The session identifier should never be stored anywhere other than the `PHPSESSID` cookie itself.

Business impact:

- A single reflected or stored XSS anywhere in the application yields full account takeover.
- Any accidental HTTP endpoint (e.g., a redirect misconfiguration) leaks sessions on the network.
- CSRF is harder to mitigate because the browser's SameSite default is not engaged.

Recommendation

1. Set the session cookie to `Secure; HttpOnly; SameSite=Lax` (or `Strict` for admin flows). In PHP: `session_set_cookie_params(['lifetime' => 0, 'path' => '/', 'secure' => true, 'httponly' => true, 'samesite' => 'Lax']);`
2. Stop including the session identifier in the `last_login` cookie — a timestamp is enough; drop `session_id` from that payload.
3. Rotate the session ID (`session_regenerate_id(true)`) on each privilege change (login already does this; make sure logout + password-change also do).

M1: Broken Object Level Authorization (BOLA) on image upload — attacker can upload images into any other user's account	
Score	6.5 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N
Target	POST https://imagesharer.xyz/images/upload
References	• https://cwe.mitre.org/data/definitions/639.html • https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/ • https://owasp.org/Top10/A01_2021-Broken_Access_Control/

Overview

The image upload endpoint reads the owner UUID from a hidden `user_id` form field sent by the client instead of the authenticated session. An authenticated attacker can set this field to any other user's UUID (which is enumerable from the public S3 bucket) and have their uploaded image recorded against that user's account.

Details

The upload form includes:

```
<input type="hidden" name="user_id" value="<the-current-user's-uuid>">
```

The server trusts the submitted `user_id` as-is. A test registered account A (`740532d8-b0f3-4cad-a69a-43a0de29a4c8`) submitted a benign JPEG with the form field `user_id=0d542b4a-75de-4dc5-ac0b-35fed0279387` (account B). The server responded `302 Found → /images/fb7b20b2-3743-4824-b119-461501096203`. Account B's dashboard subsequently showed the image as if B had uploaded it; account A did not see it. The storage object was written to `users/0d542b4a-75de-4dc5-ac0b-35fed0279387/fb7b20b2-....jpg` on S3.

Root cause confirmation: when the `user_id` field is removed, the backend throws `Undefined array key "user_id" in /var/www/html/src/UI/ImagesUiController.php:42` — meaning the controller never falls back to the authenticated session user; it strictly requires and trusts the client-sent field.

Chaining:

- Combined with the lack of CSRF protection (separate finding), the attacker can plant images under any user's account using a malicious page that posts cross-origin while the victim is logged in.
- Combined with the public S3 bucket, the attacker already knows every valid `user_id` to target.

- Combined with the loose file-upload validation (double-extension accepted; separate finding), the attacker can plant content with hostile filenames or embedded payloads under a victim's name.

Business impact:

- **Attribution attack:** an attacker can plant arbitrary (and potentially illegal or compromising) imagery under a victim's account, exposing the victim to legal action, abuse reports, or storage-quota abuse.
- Reputational damage to the service once the behaviour is public (any user can upload "as" another user).
- Loss of integrity for the entire image dataset — the "who uploaded this" field can no longer be trusted for any purpose, including moderation, forensics, or deletion-of-user-data responses.

Recommendation

1. In `ImagesUiController.php` (around line 42) and in `ImagesService.php` (around line 71), ignore any client-supplied `user_id` and read the owner UUID exclusively from the authenticated session (e.g., `$_SESSION['user_id']`).
2. Remove the hidden `user_id` field from the upload form; it serves no purpose and signals a trust boundary error.
3. Review all other POST handlers for the same pattern (profile avatar POST, any API endpoint). The avatar upload POST does not expose this field, but the server-side logic should be audited to confirm it also derives owner from the session.
4. Delete all images uploaded during the exposure window that cannot be attributed with confidence to the session user they live under; audit the `users//` key and the `users/00000000-.../` keys which indicate the controller previously accepted attacker-supplied strings.

M2: Backend trusts client-supplied `user_id` parameter and bubbles unvalidated input into database queries and storage paths	
Score	6.4 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:L/I:L/A:N
Target	POST https://imagesharer.xyz/images/upload (and almost certainly related endpoints)
References	• https://cwe.mitre.org/data/definitions/20.html • https://cwe.mitre.org/data/definitions/915.html

Overview

The upload handler forwards the client-supplied `user_id` form field directly into the SQL layer and into the S3 object key. Along with the ownership bypass already documented (BOLA finding), this permits attacker control over the storage prefix of the object, which has produced persistent artifacts in the bucket (e.g., the key `users/''/a5cebccb-...png` demonstrates `user_id='` reaching the storage path).

Details

Observed effects when tampering with the `user_id` form field:

- Valid UUID of another user → file stored under that user's prefix (see BOLA finding).
- `../../etc` → request rejected to `/images/upload` (filter catches `..`).
- `' or 1=1 --` → HTTP 500 with the PostgreSQL error `invalid input syntax for type uuid` (the DB prepared statement rejects the type-coerce, which incidentally prevents SQLi via this parameter).
- blank or missing → HTTP 500 with PHP source paths (information disclosure finding).
- The public bucket contains `users/''/...` showing that at least once in the past, a single apostrophe survived the controller and made it onto the storage path.

Business impact:

- Attackers can seed garbage keys into the bucket (`users/<attacker-chosen-prefix>/<uuid>.jpg`), causing operational mess and complicating any later "delete all my data" request.
- Even where PostgreSQL's strict UUID type rejects malformed values, the application should not depend on database type-coercion as its input validator.
- Any future query that doesn't go through the UUID-typed parameter (e.g., a free-text `WHERE email = ?` elsewhere) carries the same trust problem.

Recommendation

1. Validate every `user_id` field against the UUID format in the UI controller (`ImagesUiController.php`) BEFORE any DB or storage access. Reject with 400 on failure.

2. More importantly: do not read `user_id` from the request at all — read it from the authenticated session (see BOLA finding). This removes the whole category.
3. Delete any existing bucket keys whose prefix is not a well-formed UUID (`users/''`, `users/00000000-0000-0000-0000-000000000000/`, etc.) — they are leftover from earlier probing and have no legitimate owner.
4. Apply the same lesson to all other controllers: trust nothing from the client that duplicates server-known state.

M3: Open recursive DNS resolver on port 53	
Score	5.8 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:N/I:N/A:L
Target	51.161.131.44:53/tcp (and almost certainly 53/udp)
References	• https://cwe.mitre.org/data/definitions/406.html • https://www.cisa.gov/news-events/alerts/2014/01/17/dns-amplification-attacks

Overview

The VPS answers recursive DNS queries for arbitrary external names from any source IP on the Internet. This both exposes the host to being abused as an amplification reflector in DDoS attacks against third parties and leaks hints about internal infrastructure/queries.

Details

`dig +tcp @51.161.131.44 example.com a +rec` returned a standard answer with the `ra` (recursion available) flag set. `+norec` queries were also answered. Standard CHAOS probes (`version.bind`, `hostname.bind`) returned NXDOMAIN (good — software version not disclosed). AXFR for `imagesharer.xyz` was correctly refused.

The server appears to be a locally-running resolver (likely `systemd-resolved`, `bind9`, or `unbound`) that is unintentionally bound to the public interface rather than loopback.

Business impact:

- The VPS can be weaponised to amplify UDP-based DDoS traffic at other victims, which can get the IP (and by extension the hosting account) null-routed or rate-limited by OVH.
- Potential privacy leak if any internal names are inadvertently resolved.
- Reputation and abuse-complaint risk.

Recommendation

1. Bind the resolver to `127.0.0.1` only (e.g., `listen-on { 127.0.0.1; };` in bind9 or `DNSStubListener=no` in `systemd-resolved`).
2. Alternatively, firewall `53/tcp` and `53/udp` so the outside world cannot reach it; the application does not need to receive DNS from the Internet.
3. If a public authoritative DNS is a real requirement, split authoritative from recursive and set `recursion no;` on the authoritative instance.

M4: Password policy weak and no password-change / password-reset flow exists	
Score	5.4 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N
Target	• POST https://imagesharer.xyz/register (sets passwords with no complexity requirement) • No /password / /change-password / /forgot-password / /reset-password route exists
References	• https://owasp.org/www-project-application-security-verification-standard/ • https://pages.nist.gov/800-63-3/sp800-63b.html

Overview

Registration accepts passwords of 8+ characters with no complexity, dictionary, breached-password, or entropy checks. There is no way for a logged-in user to change their password, and no public password-reset / forgot-password route exists. Once a user's password is set it cannot be rotated — and if it is compromised, the user has no remediation path other than account abandonment.

Details

- Register form hint: `minlength="8"`, `placeholder="Min. 8 characters"`. Accepted `password=TestPassword123` without resistance; no breach check observed.
- Enumerated routes `/password`, `/change-password`, `/profile/password`, `/account/password`, `/forgot`, `/forgot-password`, `/reset`, `/reset-password` all returned 404.
- The profile page contains no password or email change UI.

Business impact:

- Users with weak or breached passwords cannot be nudged to improve them.
- Users whose password is compromised cannot defensively rotate it.
- The service cannot comply with policies that require periodic rotation or breach-triggered rotation.

Recommendation

1. Implement a `/profile/password` POST endpoint that requires the current password and sets a new one (with the same strength checks as registration). Rotate the session ID on success.
2. Implement a `/forgot-password` + `/reset-password` flow with a signed, time-limited, single-use token delivered by email.
3. Strengthen the policy: minimum 12 characters, check against a leaked-password list (Have I Been Pwned k-anonymity API), reject passwords containing the email local-part.
4. Add email verification at registration (also helps the enumeration issue above).

M5: TLS certificate missing SAN for www.imagesharer.xyz	
Score	5.4 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:L
Target	https://www.imagesharer.xyz/
References	https://datatracker.ietf.org/doc/html/rfc6125

Overview

`www.imagesharer.xyz` resolves to the same OVH VPS as the apex and is served by the same nginx instance, but the TLS certificate contains only `imagesharer.xyz` in its Subject Alternative Names. Any visitor who types `www.imagesharer.xyz` into a browser is shown a certificate error, is likely to click through or abandon the site, and may become more conditioned to accept certificate warnings in general.

Details

`openssl s_client -connect www.imagesharer.xyz:443` presents a certificate issued for `CN=imagesharer.xyz, SAN=imagesharer.xyz` — no `www` entry. All other TLS settings are strong (TLS 1.2/1.3 only, modern cipher suites, no weak algorithms per sslscan).

Business impact:

- Users mistyping with `www.` see a scary warning page; conversion loss and user training toward ignoring TLS warnings.
- The underlying nginx responds to the `www.` host, so the only broken piece is the certificate.

Recommendation

1. Issue the Let's Encrypt certificate (or whichever ACME client is in use) with both `imagesharer.xyz` and `www.imagesharer.xyz` SANs.
2. Also publish a permanent 301 redirect from `https://www.imagesharer.xyz/*` to `https://imagesharer.xyz/*` so users land on the canonical hostname.

M6: Open user enumeration via /register endpoint	
Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	POST https://imagesharer.xyz/register
References	• https://cwe.mitre.org/data/definitions/204.html • https://owasp.org/www-project-application-security-verification-standard/

Overview

POSTing a candidate email to `/register` returns a 302 with `Location: /register` if the email already exists on the service, and `Location: /dashboard` if it does not (because the new account is created and auto-logged-in). An attacker can enumerate the entire user base by submitting candidate emails and observing the redirect target.

Details

- Existing email `clawsec.a.1791171194@mailinator.com` → `302 ... Location: /register`
- New random email `notexist123456@nowhere.local` → `302 ... Location: /dashboard`

The login endpoint `/login` behaves identically for both nonexistent and incorrect-password cases (both redirect to `/login`) and does NOT leak enumeration — the asymmetry is only on the register endpoint.

An attacker who runs through a list of likely emails (customer leaks, public rosters, etc.) can learn which addresses have accounts. Combined with the lack of rate limiting (next finding), this is practical at scale.

Business impact:

- Privacy exposure (service membership can be learned without the user's consent).
- Enables targeted phishing of confirmed users.
- Enables targeted credential-stuffing (knowing an email + probable password reuse).

Recommendation

1. Make `/register` always return the same message/redirect pattern whether the email exists or not. For a confirmed existing email, send an email to that address (via the normal mail path) saying "an account already exists; here is how to reset your password" and keep the HTTP response identical to the create-new-account case.
2. Add a verification email requirement for new accounts so that creating an account cannot be used as a side-channel at all.
3. Add rate limiting (next finding) so even response-timing differences cannot be exploited at scale.

M7: Email domain lacks SPF, DMARC, DKIM, and MX records — email spoofing permitted	
Score	4.7 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:N/I:L/A:N
Target	imagesharer.xyz (DNS zone)
References	• https://cwe.mitre.org/data/definitions/290.html • https://datatracker.ietf.org/doc/html/rfc7489

Overview

`imagesharer.xyz` has no MX, SPF, DMARC, or DKIM records. Any attacker can send email appearing to come from `@imagesharer.xyz` to users or partners, with no reliable way for the receiving mail server to detect the forgery. This is especially risky if the service ever sends email (password reset, security notifications), because recipients cannot distinguish real messages from spoofed ones.

Details

Direct DNS queries returned nothing:

```
MX imagesharer.xyz          -> no answer
TXT imagesharer.xyz         -> no SPF record
TXT _dmarc.imagesharer.xyz  -> no DMARC record
TXT default._domainkey.imagesharer.xyz -> no DKIM record
```

Business impact:

- Phishing from `@imagesharer.xyz` cannot be filtered confidently by downstream mail systems.
- If the service plans to send email, any legitimate mail may land in spam.
- Reputational and legal risk if users are deceived.

Recommendation

1. Publish a hard-fail SPF record even if the domain never sends mail: `imagesharer.xyz. TXT "v=spf1 -all"`
2. Publish a reject-policy DMARC record: `_dmarc.imagesharer.xyz. TXT "v=DMARC1; p=reject; sp=reject; rua=mailto:dmarc@imagesharer.xyz"`
3. When outbound mail is introduced, add the corresponding sending-service SPF `include` and a DKIM TXT at a selector managed by the sender; keep DMARC at `p=reject`.
4. Even without inbound mail, publish an MX that returns `"."` (null MX, RFC 7505) to signal that the domain does not receive mail: `imagesharer.xyz. MX 0 "."`

M8: 7-day pre-signed URLs for public images are excessive given the bucket is public anyway	
Score	4.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N
Target	GET https://imagesharer.xyz/images/<uuid> for images with is_public=true
References	https://docs.aws.amazon.com/AmazonS3/latest/userguide/ShareObjectPreSignedURL.html

Overview

Public images are served through pre-signed URLs that are valid for 7 days (`X-Amz-Expires=604800`). Private images use 15 minutes (`X-Amz-Expires=900`), which is appropriate. The 7-day lifetime does not meaningfully help the application and leaves a long-lived valid URL in logs, referers, chat screenshots, etc.

Details

Observed with uploaded test images:

- `is_public=false` → `X-Amz-Expires=900`
- `is_public=true` → `X-Amz-Expires=604800`

The AWS access key ID (redacted in this report) and the full signature are included in every URL. While this URL is only "weakly" sensitive, 7 days is long enough that any accidental logging or reference exposure persists.

Note: because the underlying bucket is anonymously readable (see the critical finding), the pre-signed URL confidentiality is already defeated. Once the bucket is locked down, the 7-day issue becomes relevant again.

Business impact:

- Pre-signed URLs leaking into web server logs, analytics, screenshots, or chat remain valid for a week.
- Longer-than-necessary URL exposure window.

Recommendation

1. Reduce `X-Amz-Expires` for all pre-signed URLs to the shortest value that satisfies the user flow — typically 15–60 minutes even for "public" previews.
2. If genuinely long-lived public URLs are needed, use S3's `public-read` ACL on a dedicated public bucket (and only after the main bucket is locked down and only for images the user has explicitly marked public with a confirmation UX).

M9: Verbose error messages leak source file paths and database schema (information disclosure)	
Score	4.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N
Target	POST https://imagesharer.xyz/images/upload (triggers on bad user_id input)
References	• https://cwe.mitre.org/data/definitions/209.html • https://owasp.org/Top10/A05_2021-Security_Misconfiguration/

Overview

When the image upload controller receives an invalid or missing `user_id` field, the server responds with HTTP 500 and a JSON body that includes the full server-side file path, line number, and (for DB errors) the underlying PostgreSQL error with parameterised query details.

Details

Observed responses:

- Blank `user_id` field → `{"error":{"code":"INTERNAL_ERROR","message":"An unexpected error occurred.","detail":"SQLSTATE[22P02]: Invalid text representation: 7 ERROR: invalid input syntax for type uuid: \"\"\nCONTEXT: unnamed portal parameter $2 = '' in /var/www/html/src/modules/images/ImagesService.php:71"}}`
- Missing `user_id` field → `{"error":{"code":"INTERNAL_ERROR","message":"An unexpected error occurred.","detail":"Undefined array key \"user_id\" in /var/www/html/src/UI/ImagesUiController.php:42"}}`

These responses disclose:

- Document root (`/var/www/html/`)
- Internal module structure (`src/modules/images/`, `src/UI/`)
- Specific class/file names (`ImagesService.php`, `ImagesUiController.php`)
- PostgreSQL as the backend database
- Parameterised query index (`$2`) — on the positive side, this also confirms prepared statements are in use (reducing SQL injection risk via that parameter)

Business impact:

- Dramatically shortens an attacker's reconnaissance phase.
- Confirms technology stack and project layout, enabling targeted exploitation of known CVEs or framework behaviours.
- Can expose secrets if any ever get stringified into error paths.

Recommendation

1. Set the application to production error-handling mode: catch all unhandled exceptions in a single boundary handler that returns a generic opaque error reference (e.g., `{"error":{"code":"INTERNAL_ERROR","reference":"<uuid>"}}`) and logs the detail server-side only.
2. In PHP, set `display_errors = Off`, `display_startup_errors = Off`, `log_errors = On`, `error_reporting = E_ALL & ~E_DEPRECATED & ~E_STRICT` for production.
3. Validate inputs (`user_id` must be a UUID; missing fields should fail validation long before touching the DB) and return `400 Bad Request` with a safe message instead of letting exceptions bubble up.

M10: Weak file-upload validation on /images/upload — polyglot and double-extension files accepted	
Score	4.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N
Target	POST https://imagesharer.xyz/images/upload
References	• https://cwe.mitre.org/data/definitions/434.html • https://owasp.org/www-project-application-security-verification-standard/

Overview

The image upload endpoint accepts files whose filenames end in `.jpg.php` and whose body contains JPEG magic bytes followed by PHP source code. The avatar upload endpoint rejects the same files, indicating the two upload paths use different validators and the images path is weaker. The uploads are stored on S3 and re-keyed to `<uuid>.jpg`, which stops direct code execution — but hostile content still enters user storage, and the server-side renaming is the only thing between a bad upload and a persistent exploit.

Details

Test matrix (`/images/upload`, authenticated as account A, with the correct `user_id`):

Filename sent	Content-Type	Content	Result
`xss.svg`	`image/svg+xml`	SVG with `<script>`	302 → /images/upload (rejected)
`shell.php`	`application/x-php`	`<?php phpinfo(); ?>`	302 → /images/upload (rejected)
`shell.jpg.php`	`image/jpeg`	pure PHP	302 → /images/upload (rejected)
`magic_php.jpg`	`image/jpeg`	JPEG header + `<?php ... ?>`	302 → /images/ (**ACCEPTED**)
`double_ext.jpg.php`	`image/jpeg`	JPEG header + `<?php ... ?>`	302 → /images/ (**ACCEPTED**)

The accepted files were renamed to `<image-uuid>.jpg` on S3, which prevents direct `.php` execution. But the application endpoint `/images/<uuid>` still serves these to users as the application's "image," meaning hostile content now lives under a victim's account (chainable with the BOLA finding) and will be served to any viewer with a believable preview thumbnail.

The avatar endpoint rejected all of the same test files, proving the application has a stronger validator — it just is not applied uniformly.

Business impact:

- The weaker image-upload path accepts polyglots that can carry secondary payloads (abusive content hidden in images, data exfiltration via image steganography, EXIF-stored payloads).
- Combined with BOLA and CSRF, an attacker can plant hostile files under a victim's account without the victim's awareness.
- Any future change that treats uploaded files by filename (image resizers, antivirus scanners, OCR, download endpoints) is at risk.

Recommendation

1. Reuse the exact validator used by `/profile/avatar` on `/images/upload`.
2. Validate uploads server-side by (a) MIME type sniffing of the first N bytes (`finfo_file`), (b) image parsing via GD/Imagick to confirm it is a valid image of the expected type, and (c) strict allow-list of extensions.
3. Always re-encode uploaded images (decode → re-encode as JPEG/PNG/WebP) to strip EXIF / embedded payloads / polyglot tails. This also prevents EXIF-based data exfiltration and some stored-XSS techniques.
4. Enforce the type on the S3 object metadata (`Content-Type: image/*`, `Content-Disposition: inline`, consider `X-Content-Type-Options: nosniff` at the CDN layer).

M11: Missing security headers (HSTS, Content-Security-Policy, Permissions-Policy)	
Score	4.2 (Medium)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:L/A:N
Target	all responses from https://imagesharer.xyz/
References	• https://owasp.org/www-project-secure-headers/ • https://developer.mozilla.org/en-US/docs/Web/Security

Overview

The web application returns some good headers (`X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, `X-XSS-Protection`) but omits HSTS, Content-Security-Policy, and Permissions-Policy. HSTS absence allows a first-visit downgrade to HTTP, CSP absence removes a critical mitigation against XSS, and Permissions-Policy absence lets any embedded resource request camera, microphone, geolocation, etc.

Details

Observed response headers (consistent across the site):

```
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
```

Missing:

- `Strict-Transport-Security`
- `Content-Security-Policy`
- `Permissions-Policy`
- `Cross-Origin-Opener-Policy` / `Cross-Origin-Embedder-Policy` / `Cross-Origin-Resource-Policy` (defence-in-depth)

Business impact:

- A user who types `imagesharer.xyz` into their browser bar lands on HTTP first; without HSTS, a network attacker can keep them on HTTP.
- Any XSS vulnerability (none proven exploitable here, but the attack surface is large) could be dramatically mitigated by a tight CSP.
- Future features that embed third-party content inherit the (lack of) policy.

Recommendation

1. Add on all HTML responses:
   - `Strict-Transport-Security: max-age=63072000; includeSubDomains; preload` (consider submitting the domain to the HSTS preload list after a short observation window).
   - `Content-Security-Policy: default-src 'self'; img-src 'self' https://*.s3.ap-southeast-2.amazonaws.com data:; script-src 'self'; style-src 'self' 'unsafe-inline'; object-src 'none'; frame-ancestors 'none'; base-uri 'self'; form-action 'self';` (tune to the real asset set).
   - `Permissions-Policy: geolocation=(), microphone=(), camera=(), payment=()` (deny everything the app doesn't need).
2. Add a CAA record on the apex domain restricting issuance to the current CA (Let's Encrypt): `imagesharer.xyz. CAA 0 issue "letsencrypt.org"`.
3. Drop the deprecated `X-XSS-Protection` header (it is removed from modern browsers) and replace with a CSP.

I1: API authentication is functionally unreachable because the Authorization header is stripped before PHP (configuration bug)	
Score	0.0 (Info)
Vector string	N/A
Target	/api/v1/* on both nginx (443) and Apache (8080)
References	• https://httpd.apache.org/docs/2.4/mod/core.html#cgipassauth • https://www.php.net/manual/en/reserved.variables.server.php

Overview

The REST API accepts a POST to `/api/v1/auth/login`, returns a JWT, and advertises a Bearer-token flow. In practice no authenticated API request succeeds, because `Authorization: Bearer <token>` is stripped by the server stack before PHP sees it; the API always answers `401 UNAUTHORIZED – Authentication token is required`. This is a configuration/functionality bug rather than an attacker-exploitable vulnerability, but it means the whole `/api/v1/*` surface is dead and any security claims about it are unverifiable.

Details

- `POST /api/v1/auth/login` with valid credentials returns an HS256 JWT (payload: `{"iss":"imager","sub":"<user-uuid>","iat":<ts>,"exp":<ts+86400>}`), 24-hour lifetime.
- Replaying that token in `Authorization: Bearer`, `authorization: bearer`, `X-Auth-Token`, `?token=`, `Proxy-Authorization`, and `Cookie: token=` all yield the same 401 `Authentication token is required.`
- This behaviour is observed BOTH on the nginx-fronted 443 and on the direct Apache 8080 — so this is PHP configuration (`CGIPassAuth off`-like default) rather than nginx.

Business impact:

- The API is effectively non-functional for any client that uses it, which has no security consequence by itself but points to an untested code path. Once fixed, the API would need to be re-audited (JWT secret strength, revocation, scopes, etc.).

Recommendation

1. In Apache, add `SetEnvIf Authorization "(.*)" HTTP_AUTHORIZATION=$1` (or `CGIPassAuth On` in a `<Directory>` block) so the `Authorization` header reaches PHP.
2. In nginx with FastCGI, add `fastcgi_pass_header Authorization;`.

3. After fixing, verify the JWT verification path:
   - Confirm `alg: none` is rejected.
   - Confirm the HS256 secret is high entropy ($\geq$ 256 bits of randomness, not a dictionary word) — the current secret could not be brute-forced in this engagement (not permitted) but should be reviewed.
   - Confirm the API also derives authorization from the JWT `sub` and never from a client-supplied `user_id` field (see BOLA finding) — any re-enabled API endpoint should pass the same ownership tests.

5 Disclaimer

The purpose of this research project is to help evaluating the seamlessness, effectiveness and cost-benefit ratio of autonomously performing vulnerabilty assessment using agentic AI. I do not take any responsibility upon any misuse or abuse of such tool where it can face significant legal actions.

A Appendix

A.1 Static Appendix Section